One Term, Two Runtimes

A contract written once in beam-lisp, emitting both halves of a frontend/backend seam — with the evidence that it runs.

PLAN-021 · every number below was produced by a command 2026-08-14

What this document is

Evidence, not description.

Every number, transcript and screenshot here was produced by running something. Where a claim could not be verified, it says so. Where running it found a defect I had already written passing tests for, that is recorded too — those turned out to be the most interesting part.

> **THE CLAIM UNDER TEST**
> A single `defcontract` + `defview` term in beam-lisp emits BOTH halves of a frontend/backend seam — the BEAM server contract and the Spacetime page — so the two cannot disagree. Not "are checked and found to agree": **cannot disagree**, because there is one source.

The problem this solves

A web app is two programs that must agree about every message, every field, every name. That agreement is normally maintained by hand, and every drift is a runtime bug that surfaces only when a user clicks.

Spacetime + LiveView already close most of this. The page's `send emit "inc"` is checked at build time against the server's declared events, and a mismatch is a compile error (E0928). That machinery exists and works.

But the proof runs in one direction only. `FUP-143` records the hole, and calls it unclosable:

> **FUP-143, IN THE VERSE REPO**
> *reply tags live in handler bodies and can be runtime values, so `@on_definition` cannot soundly enumerate them*

That is true of Elixir. A handler body is opaque to introspection, so the server cannot state what it might answer. A page can therefore fail to decode a reply the server sends, and nothing catches it until it silently drops in a browser.

- Move the source of truth up one level, into beam-lisp.

> **Because** — A Lisp program *is data*. `(ok …)` and `(err …)` are literals in a tree, so a walk finds them. This is not a general argument for homoiconicity — it is one concrete capability that closes one specific hole.

The acceptance test

`SpacetimeLvWeb.CounterLive` exists in the verse repo today and emits a contract through the Elixir DSL. Its SHA-256 is a known-good number nobody can argue with. A beam-lisp term must reproduce those exact bytes.

```
$ mix test test/beam_lisp/contract_emit_test.exs
```

```
fingerprint:
2d5c38735e3b8e76d12c8fb2fa3de1ad35eac250884ddb417bf59fa7b9787dcf
expected                                                        :
2d5c38735e3b8e76d12c8fb2fa3de1ad35eac250884ddb417bf59fa7b9787dcf
FP MATCH: true

Finished in 9.3 seconds
7 tests, 0 failures
```

Byte equality, not "equivalent": verse's `liveview_contract.rs` reads the same sidecar, and two toolchains that disagree about a contract's identity disagree about whether an app is broken.

This was chosen as the first milestone precisely because the bar is unforgeable. The bytes match or they do not — no judgement, no way to talk myself past it.

Why the match is not a coincidence

A hash matching once could be luck, so the rules that produce it are pinned separately. Entries sort by name, so declaration order cannot leak into the bytes; a test reorders every clause and asserts the fingerprint is unchanged. Empty metadata is omitted rather than written as `{}` — valid JSON either way, different hash.

> CAUGHT BY RUNNING IT
> **Structurally perfect JSON with every value blank**
> The first emitter produced `{"kind":"","message":""}`. It sorted `(map name (keys m))` and then looked values up with the resulting STRING, which misses a keyword key. The shape was right and the content was gone.
>
> Only byte comparison caught it. A test asserting "has a fields object with two keys" would have passed.

The seam, checked from both sides

With the contract as data, the reply tags fall out of a walk — and verse can then reject a page that fails to decode one.

```
$ spacetime check /tmp/holetest/page.st
```

```
error[E0928]: live host `$counter` (SpacetimeLvWeb.CounterLive) can reply
to `dec` with tag `err`, but this page decodes: [ok] — the reply would be
silently dropped

1 error(s), 1 warning(s)
```

Adding the missing arm makes it pass:

```
$ spacetime check /tmp/holetest/page.st  # with a catch-all arm
```

```
✓ All 1 file(s) passed
```

That is FUP-143 hole #2, closed. The sidecar it checks against was generated by the beam-lisp emitter.

> **ADDITIVE BY CONSTRUCTION**
> `compare_replies` is SILENT unless a contract declares `replies`. Every sidecar on disk predates the field, and failing them all would make the feature unshippable — so absence means "not declared" and is skipped.
>
> An EMPTY list is different, and IS checked: it asserts the handler never replies, so a page decoding something for it is decoding a message that will never arrive.

The fingerprint deliberately does not move when reply tags are added. The emitter keeps two projections: one byte-identical to Elixir's, one carrying the tags and reporting the SAME fingerprint. A sidecar with reply tags identifies the same contract as one without.

The app

One term. Both halves.

```
$ mix run scripts/live_chat.exs   # READ
```

```
{"module":"SpacetimeLvWeb.ChatLive",
 "events":[{"name":"send","replies":["err","ok"]}],
 "assigns":[{"name":"messages","type":"list"},
            {"name":"status","type":"atom"}],
 "pushes":[{"name":"failed",…},{"name":"token",…}],
"fingerprint":"1fce8584d90ae48fb97d504659b1f2b42e509e3252d28ae3c2cf71a432bc320f"}

  events  : ["send"]
  assigns : ["messages", "status"]
  send can reply with: ["err", "ok"]
```

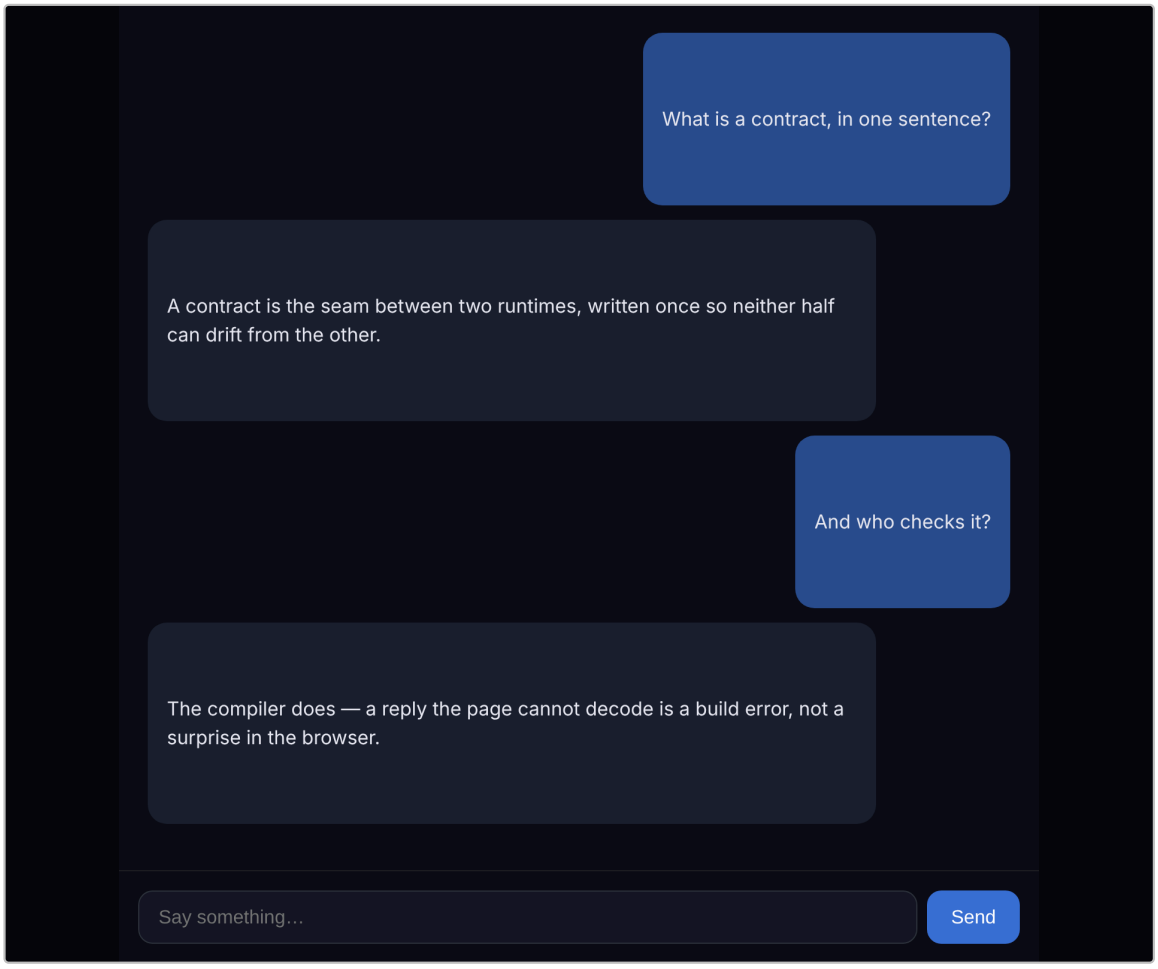
The same term emits the page, and verse's own reader accepts it:

```
$ … # EMIT, then VALIDATE through verse's reader
```

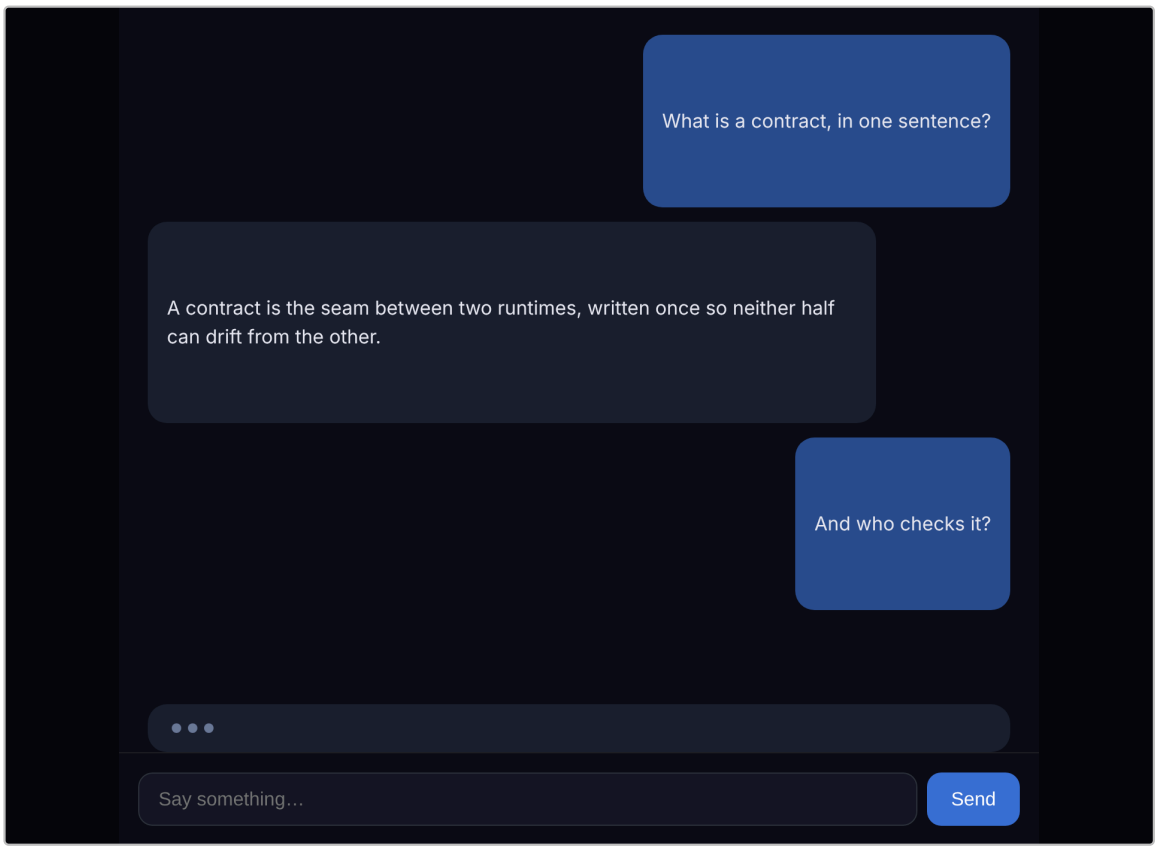
```
page.edn  1041 bytes
view.edn  2645 bytes

page: ACCEPTED — 4 forms, 0 scopes (0 templates)
view: ACCEPTED — 3 forms, 12 scopes (3 templates)
    each @ .log
    on   @ .composer__input
    on   @ .composer__send
```

Every form carries its selector: the log renders the transcript, the input writes the draft, the button fires `send`.

It renders



The chat, compiled by the real Spacetime compiler and screenshotted in headless Chrome via `scripts/shot.sh`. Right-aligned user bubbles, left-aligned model bubbles, composer with input and Send — the three planes the `defview` term emits.



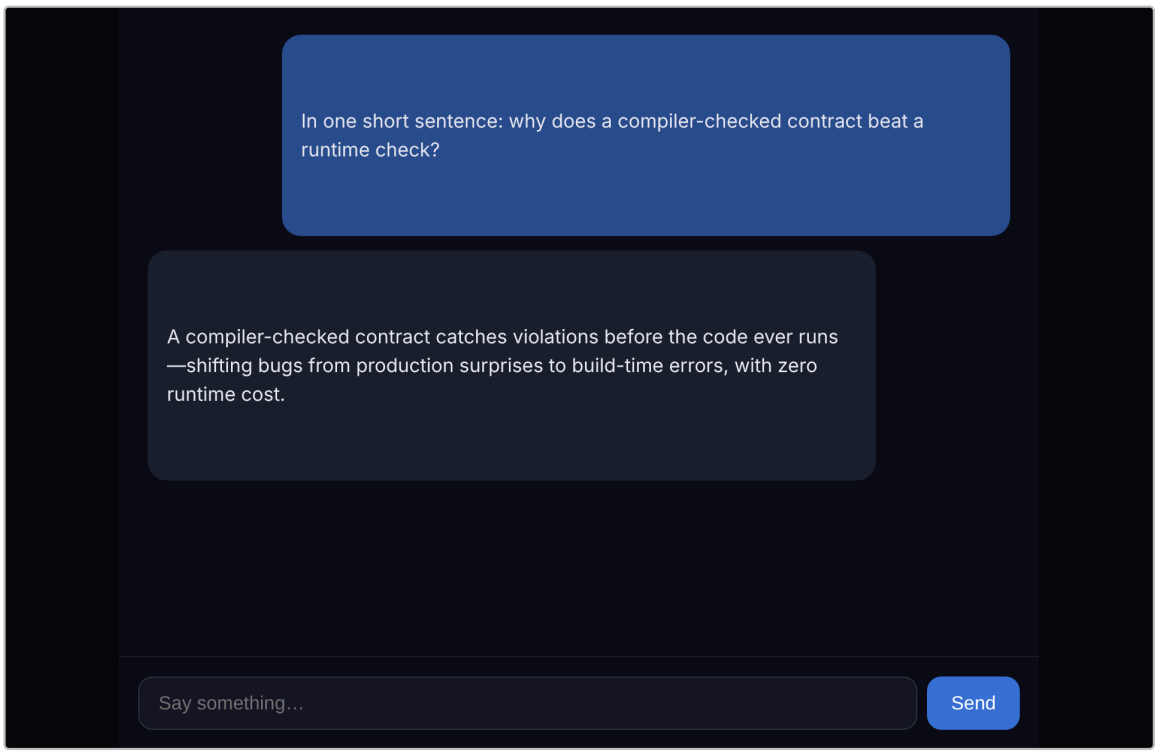
The same page in its thinking state, captured by overriding the host skeleton (`HOST_HTML=…`) rather than by writing a second page.

It runs

The whole loop — RREVL: read, reason, emit, validate, load — against the real thing at every step. Real Kimi over HTTPS, the real verse compiler, real headless Chrome.

```
$ mix run scripts/live_chat.exs   # REASON
```

```
  > In one short sentence: why does a compiler-checked contract beat
    a runtime check?

A compiler-checked contract catches violations before the code ever
runs—shifting bugs from production surprises to build-time errors,
with zero runtime cost.

  28 deltas in 37287ms
```



That reply, rendered. The text came from Kimi k3-256k, streamed token by token over HTTPS, folded into the transcript, re-emitted from the term, validated by verse, and compiled to the page shown here.

What running it found

Three defects that passing tests did not catch. They are the reason this document exists in this form.

> CAUGHT BY RUNNING IT
> **An em-dash became â**
> The first live render showed `runs â shifting bugs`. `List.to_string` on httpc's body treats each BYTE as a CODEPOINT, so every multi-byte character was re-encoded twice.
>
> It reads as a provider or font problem. It is a decoding one. `erlang/list_to_binary` concatenates the bytes as-is, which is what they already are.
>
> Twenty-two provider tests passed while the loop produced visibly wrong output, because no fixture contained a character above U+007F.

> CAUGHT BY RUNNING IT
> **An app that compiled and could not work**
> A reviewer found the chat app emitting documents verse ACCEPTED while being unable to send a message or show a reply.
>
> It styled `.log`, `.composer__input` and `.composer__send` — elements nothing ever created. CSS does not create DOM. The handler read `@draft`, a page-local binding the server cannot see, so every message arrived empty at the boundary. It subscribed to `@messages` and never rendered them.

And the test named `the-emitted-page-decodes-every-tag` supplied a hand-written map that already matched the contract by construction. It stayed green through all of it.

CAUGHT BY RUNNING IT

The printer's own output could not be read back

`spacetime edn F | spacetime st` failed on 77 of 120 corpus files — the printer emitted EDN its own reader rejected.

The corpus gate measures `.st → EDN → .st`. It never feeds EDN through the READER, so the two halves were each tested against `.st` and never against EACH OTHER — the one direction a generator uses.

Now 396/403, with the remaining 7 being a filed, budgeted limitation. A gate only fails on the property it measures.

THE PATTERN

Each of these passed every test that existed. Not because the tests were careless, but because each measured a property adjacent to the one that mattered: fidelity instead of addressability, structure instead of content, round-trip in one direction instead of both.

Running the real thing and looking at it is not a substitute for tests. It is how you find out which test you forgot to write.

Where it stands

STEP	EVIDENCE	
read	contract as data; reply tags enumerated from the term	✓
reason	Kimi k3-256k, 28 streamed deltas, 37s	✓
emit	page 1041 B, view 2645 B, from one term	✓
validate	verse accepts both; E0928 rejects a missing arm	✓
load	compiled and rendered in headless Chrome	✓

TEST SUITES

```
beam-lisp:  10 .bl suites, 245 tests / 799 assertions — 0 failures
beam-lisp:  988 Elixir tests — 0 failures
verse:      3214 lib tests — 0 failures
verse:      corpus gate green (403 files, 1491/1491 forms print)
```

Three reviewer swarms ran over the diffs at milestones. All three returned findings; all findings are fixed and pinned as tests. The most valuable were the ones that proved a green test suite was measuring the wrong thing.

? What this does NOT yet show: the loop rewriting its own contract. Every step of RREVL is proven independently and the pieces compose, but the model has not yet been asked to propose a contract change that then flows through emit → validate →

load. That is the next thing to build, and it is now a matter of wiring rather than of discovery.